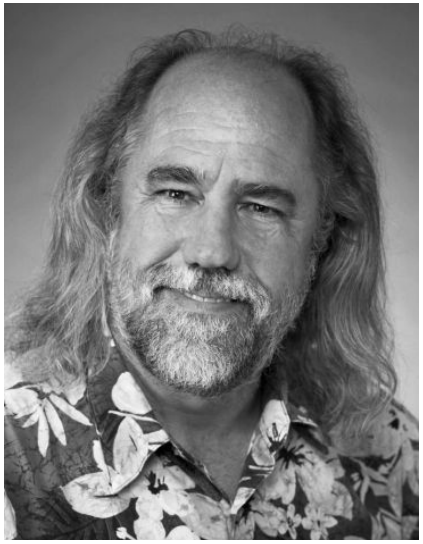
Software Engineering

UML – Unified Modeling Language

Prof. Dr.-Ing. Sven Graupner



Grady Booch, IBM



Gliederung

1. Motivation
   - Gute Bsp. aus einer Abschlussarbeit
2. Modellbildung, Modellierung von Software
3. Visuelle Darstellung von Software
   - Schlechte Beispiele
4. UML – Unified Modelling Language
   - Ursprung, Entwicklung, zwei Schulen OOSE, OMT.
5. UML Diagramm-Arten
   - Strukturdiagramme: Klassen-, Komponenten-Diagramm.
   - Verhaltensdiagramme: Use Case-, Aktivitäts-, Zustands-, Sequenz-Diagramm.
6. UML Ressourcen

1 Motivation

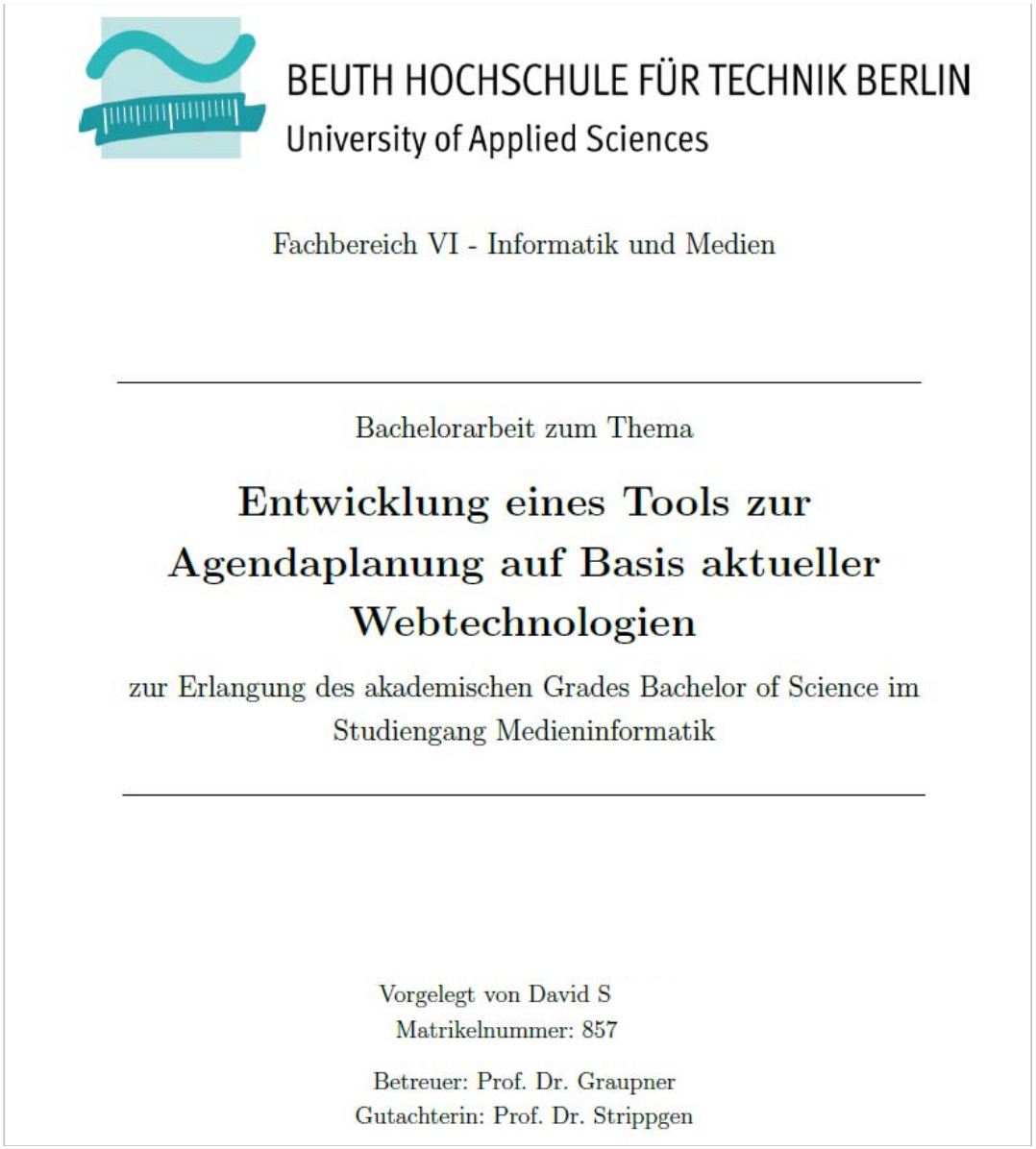
- UML ist die standardisierte Form der visuellen Darstellung von Software.
- Ein Software Engineer muss Anforderungen (Use Cases, User Stories) sowie Software (Struktur, Funktionsweise) visuell zu anderen im Team oder zu Außenstehenden kommunizieren können.
- Code ist aufgrund des Umfangs dafür oft ungeeignet.
- Code bekommt eine bessere Struktur, wenn vorher alle im Team „die gleichen Bilder im Kopf haben“.
- Das ist auch in der Abschlussarbeit wichtig.

1.1 Motivation

Beispiel einer Abschlussarbeit

in der Medien-Informatik.

(2020, bei SAP)



BEUTH HOCHSCHULE FÜR TECHNIK BERLIN
University of Applied Sciences

Fachbereich VI - Informatik und Medien

Bachelorarbeit zum Thema

Entwicklung eines Tools zur Agendaplanung auf Basis aktueller Webtechnologien

zur Erlangung des akademischen Grades Bachelor of Science im Studiengang Medieninformatik

Vorgelegt von David S
Matrikelnummer: 857

Betreuer: Prof. Dr. Graupner
Gutachterin: Prof. Dr. Strippgen

1.2 Use Cases

...skizzieren die beteiligten Rollen (Akteure) und deren erwartete Handlungen.

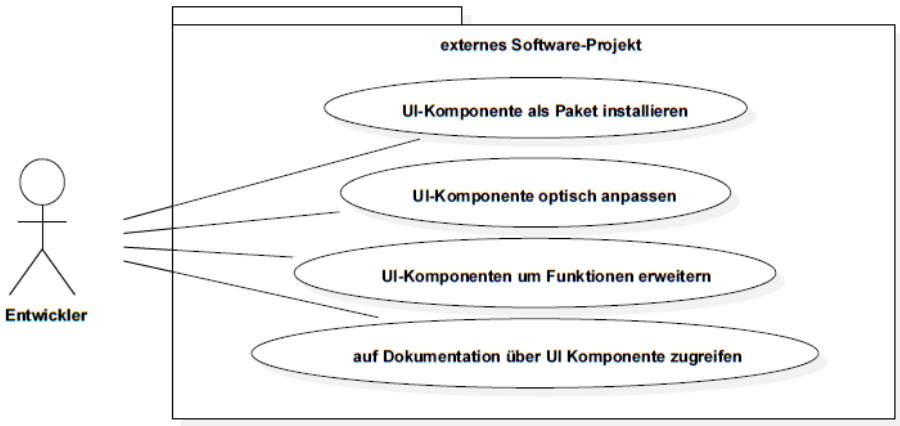
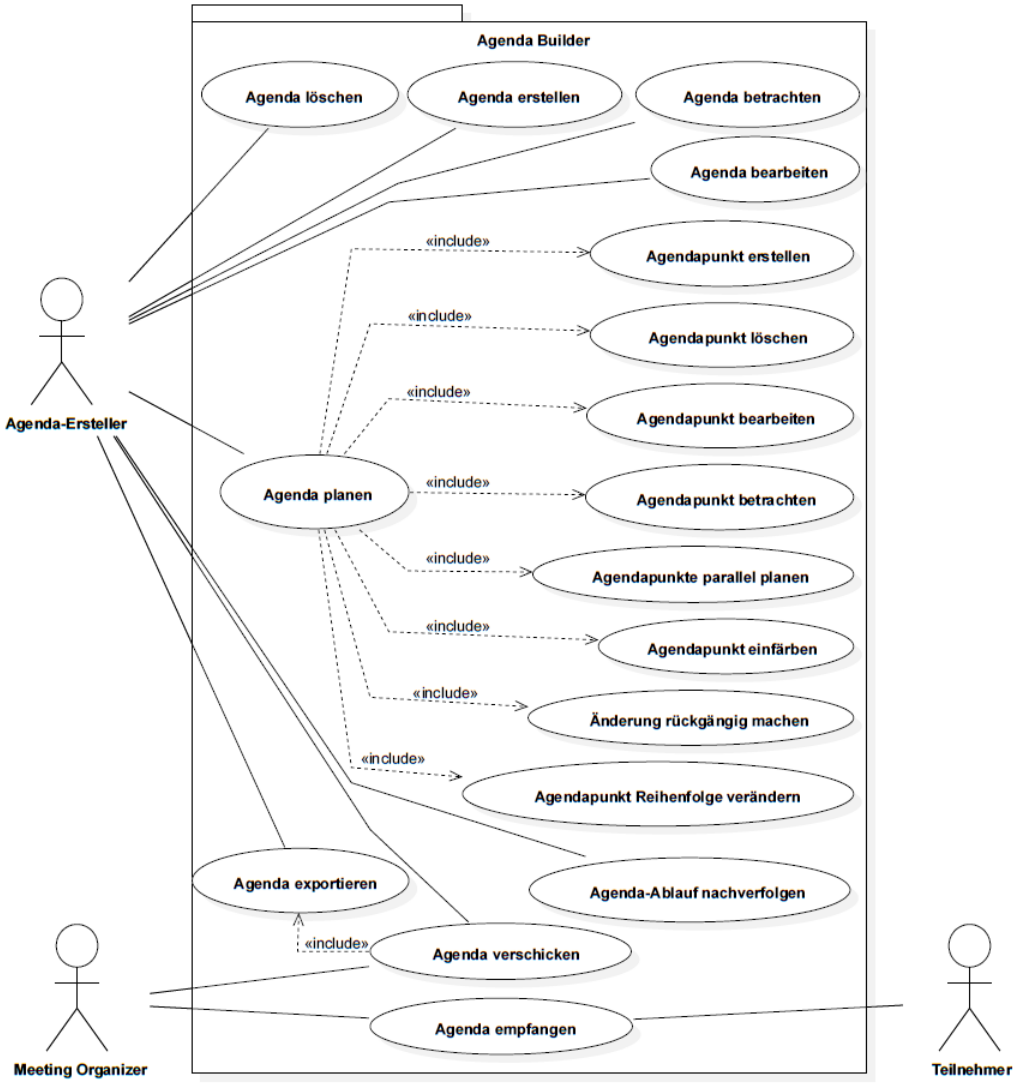


Abbildung 11: UML-Use-Case-Diagramm der Anwendungsfälle für alle Rollen.



1.3 Komponenten

...stellen die Struktur des Systems mit seinen Bestandteilen, Schnittstellen und Verbindungen dar.

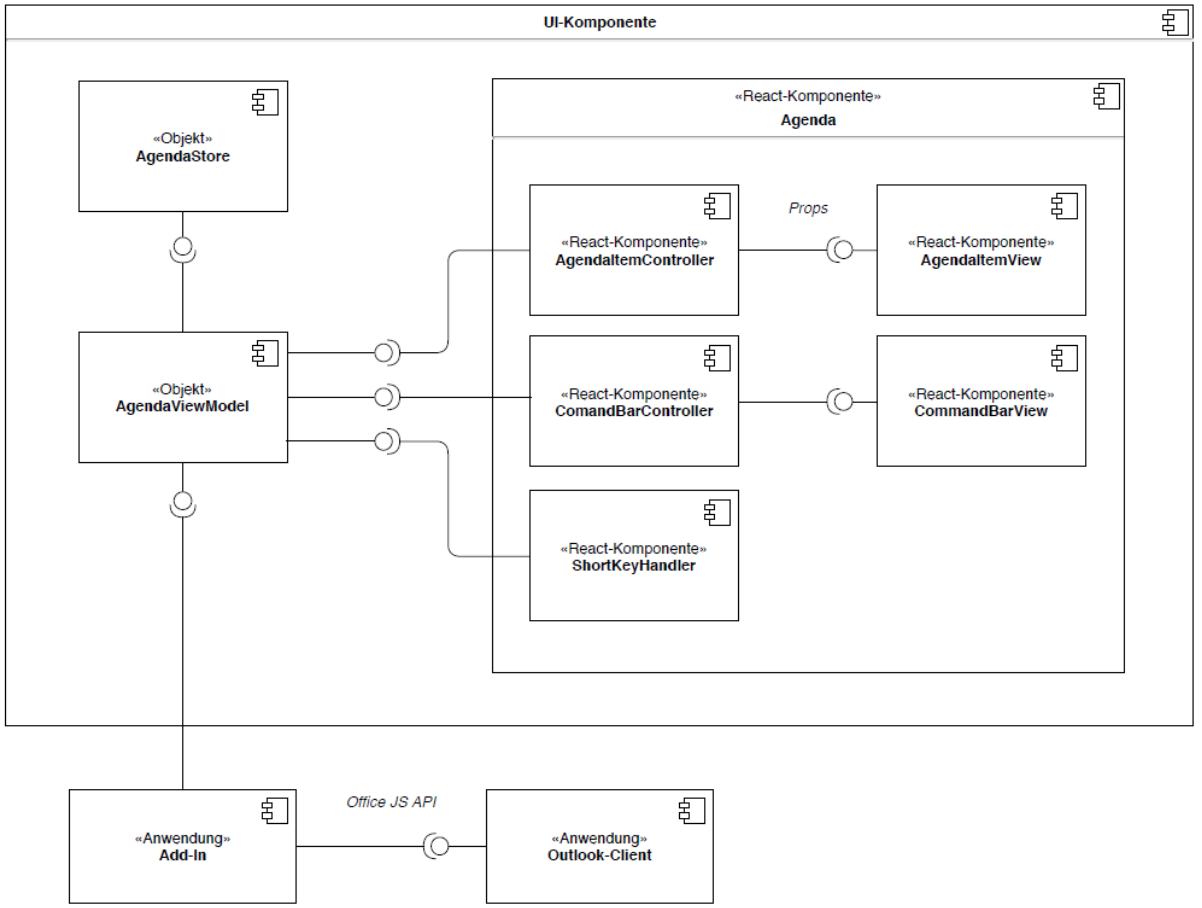


Abbildung 20: UML-Komponentendiagramm des umgesetzten MVVM-Entwurfsmusters

1.4 Datenstrukturen (als Klassendiagramm)

...zeigen die wesentlichen, vom System verwalteten Informationen.

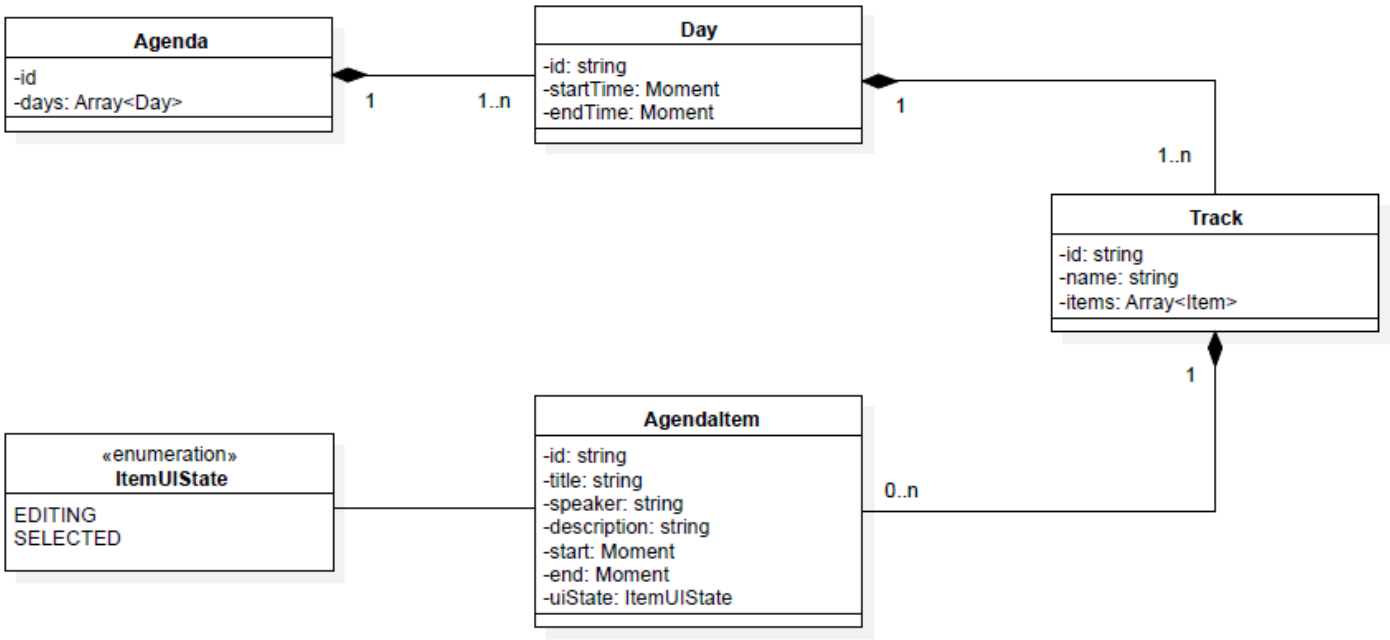


Abbildung 18: Datenmodell als UML-Klassendiagramm, Operationen bzw. Methoden sind nicht dargestellt.

1.5 Interaktionen (Sequenzdiagram)

...zeigen die wesentlichen Interaktionen zwischen den Komponenten.

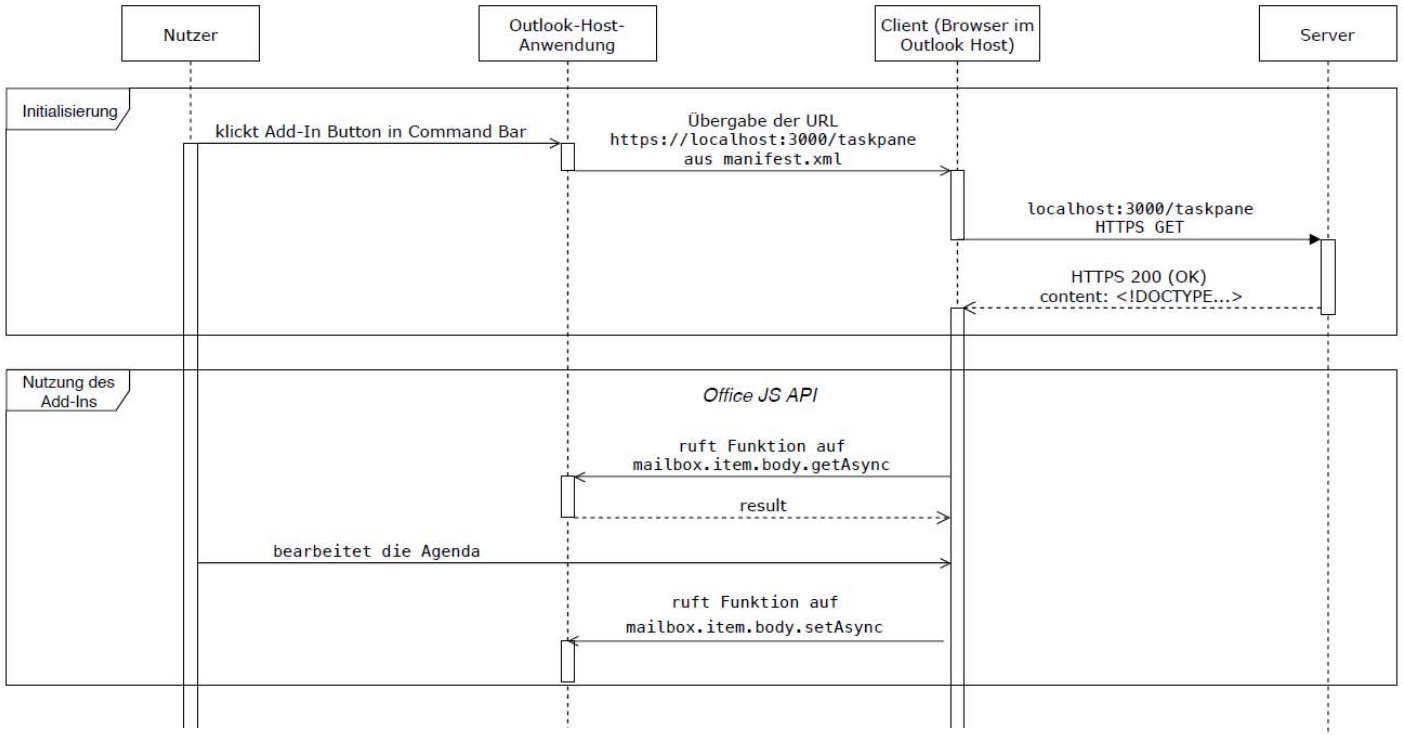


Abbildung 17: UML-Sequenzdiagramm zur Darstellung des Datenaustausches.

2 Modellbildung, Modellierung

Ein **Modell** ist ein vereinfachtes (abstrahiertes) **Abbild** der Wirklichkeit oder einer gewünschten Wirklichkeit.

Modellierung ist der Prozess der **Herstellung eines Modells** durch:

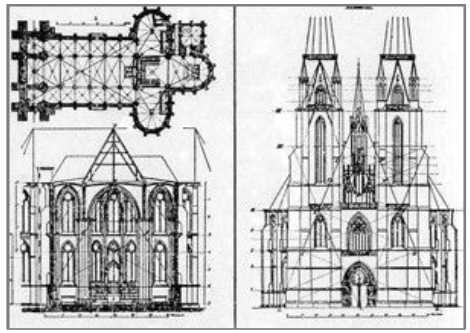
- **Analyse:**
  - Reduktion, Weglassen von Details.
  - Dekomposition, Zerlegung in Teile, Ansichten.
  - Abgrenzung, Definition von Grenzen.
- **Synthese** – Schaffung von etwas Neuem durch:
  - Abstraktion, gedankliche Verallgemeinerung, Einführung neuer Konzepte und Begriffe.
  - Aggregation, Zusammenfügen von Teilen zu einem neuen Ganzen.



2.1 Gegenständliche Modelle

Gegenständliche Modelle bilden reale oder real zu schaffende Gegenstände ab. Sie werden seit langem in der Produktion, in der Architektur oder im Bauwesen eingesetzt.









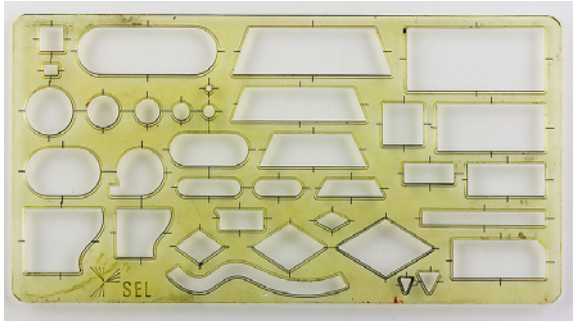
Ideen → Modelle → Herstellung → Realität

2.2 Frühe Modelle für Software

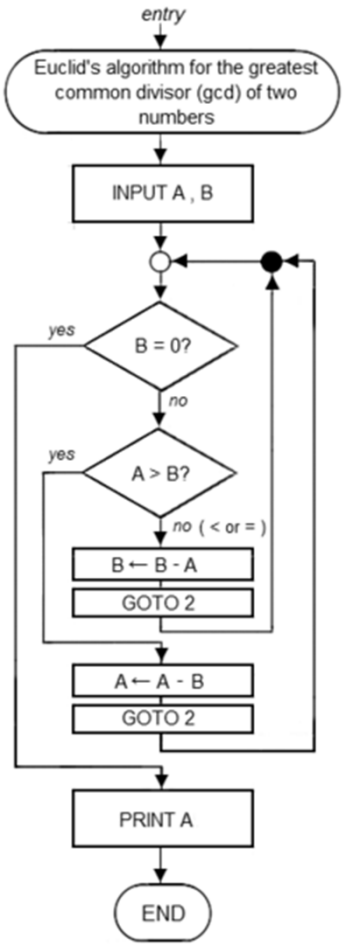
Programmablaufplan (PAP), Flussdiagramm (engl. flowchart) ist eine grafische Darstellung eines Programmablaufs.

Die Symbole sind in der DIN 66001 genormt.

Zeichenschablone für PAP:



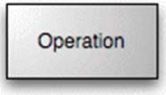
Euklid Algorithmus zur Berechnung des kleinsten gemeinsamen Vielfachen kgV als PAP.



6.4.1: Kreis; Oval / Rechteck mit gerundeten Ecken: Terminator



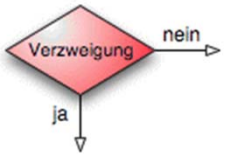
6.1.1: Rechteck: Operation (Tätigkeit)



7.2.4: Rechteck mit doppelten, vertikalen Linien: Unterprogramm ausführen



6.1.3: Raute: Verzweigung / Entscheidungen



6.2.1: Parallelogramm: Ein- und Ausgabe (ist in der DIN 66001 von 1982 zwar definiert, soll jedoch nicht für PAs verwendet werden)[2]



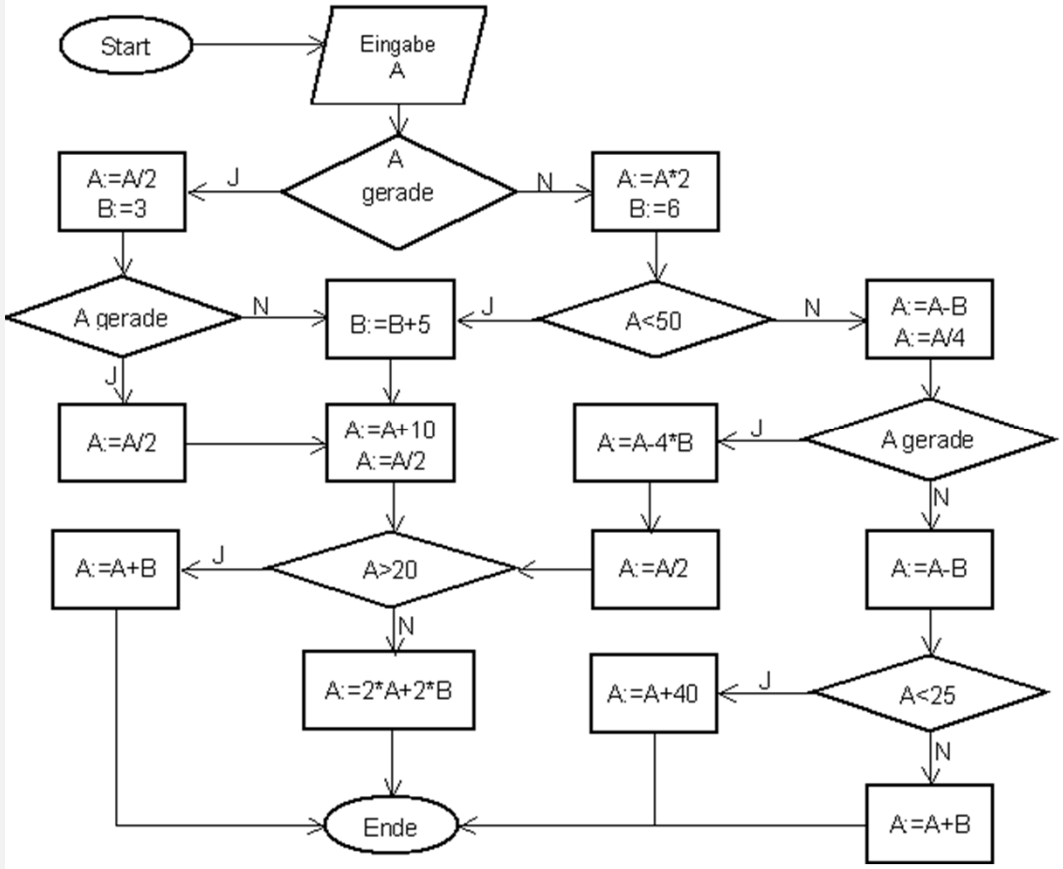
https://de.wikipedia.org/wiki/Kleinstes_gemeinsames_Vielfaches
https://www.lernhelfer.de/schuelerlexikon/mathematik/artikel/euklidischer-algorithmus

2.2.1 Das Problem von PAP

PAP Rätsel

- Kann man abschätzen, ob der Ablauf zu einem Ende kommt (terminiert)?
- Können Sie ein Ergebnis vorhersagen?
- Können Sie dieses Programm in Java implementieren?

Das Problem von PAP: Es wird keine strukturierte Programmierung (ohne goto) erzwungen, wie es in modernen Programmiersprachen heute üblich ist, z.B. in Java.



2.2.2 Unstrukturiertheit

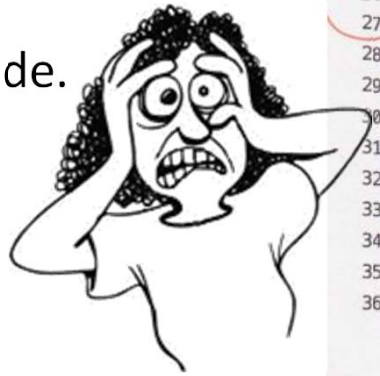
Ältere Sprachen bzw. Assembler verwenden Sprunganweisungen: **goto <Ziel-Marke>** .

im Beispiel:

- line 26: **go to 22** ; springe zu Marke 22
- line 17: multi-part Verzweigung

  IF (<expr>) lt0, eq0, gt0

Das Resultat ist „Spaghetti“-Code.



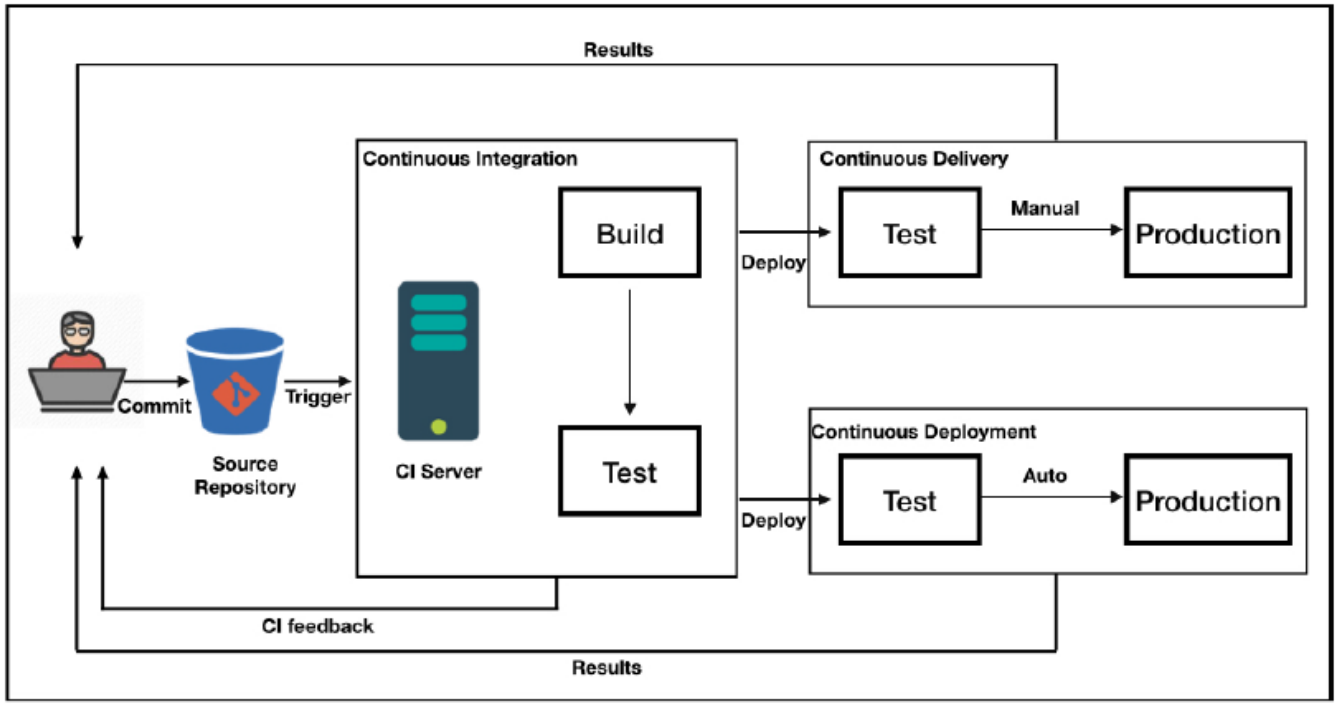
```
1  C     A weird program for calculating Pi written in Fortran.
2  C     From: Fink, D.G., Computers and the Human Mind, Anchor
3                                                        Books, 1966.
4        PROGRAM PI
5        DIMENSION TERM(100)
6        N=1
7     3  TERM(N)=((-1)**(N+1))*(4./(2.*N-1.))
8        N=N+1
9        IF (N-101) 3,6,6
10    6  N=1
11    7  SUM98 = SUM98+TERM(N)
12       WRITE(*,28) N, TERM(N)
13       N=N+1
14       IF (N-99) 7, 11, 11
15    11 SUM99=SUM98+TERM(N)
16       SUM100=SUM99+TERM(N+1)
17       IF (SUM98-3.141592) 14,23,23
18    14 IF (SUM99-3.141592) 23,23,15
19    15 IF (SUM100-3.141592) 16,23,23
20    16 AV89=(SUM98+SUM99)/2.
21       AV90=(SUM99+SUM100)/2.
22       COMANS=(AV89+AV90)/2.
23       IF (COMANS-3.1415920) 21,19,19
24    19 IF (COMANS-3.1415930) 20,21,21
25    20 WRITE(*,26)
26       GO TO 22
27    21 WRITE(*,27) COMANS
28    22 STOP
29    23 WRITE(*,25)
30       GO TO 22
31    25 FORMAT('ERROR IN MAGNITUDE OF SUM')
32    26 FORMAT('PROBLEM SOLVED')
33    27 FORMAT('PROBLEM UNSOLVED', F14.6)
34    28 FORMAT(I3, F14.6)
35       END
36
```

3 Visuelle Darstellung von Software

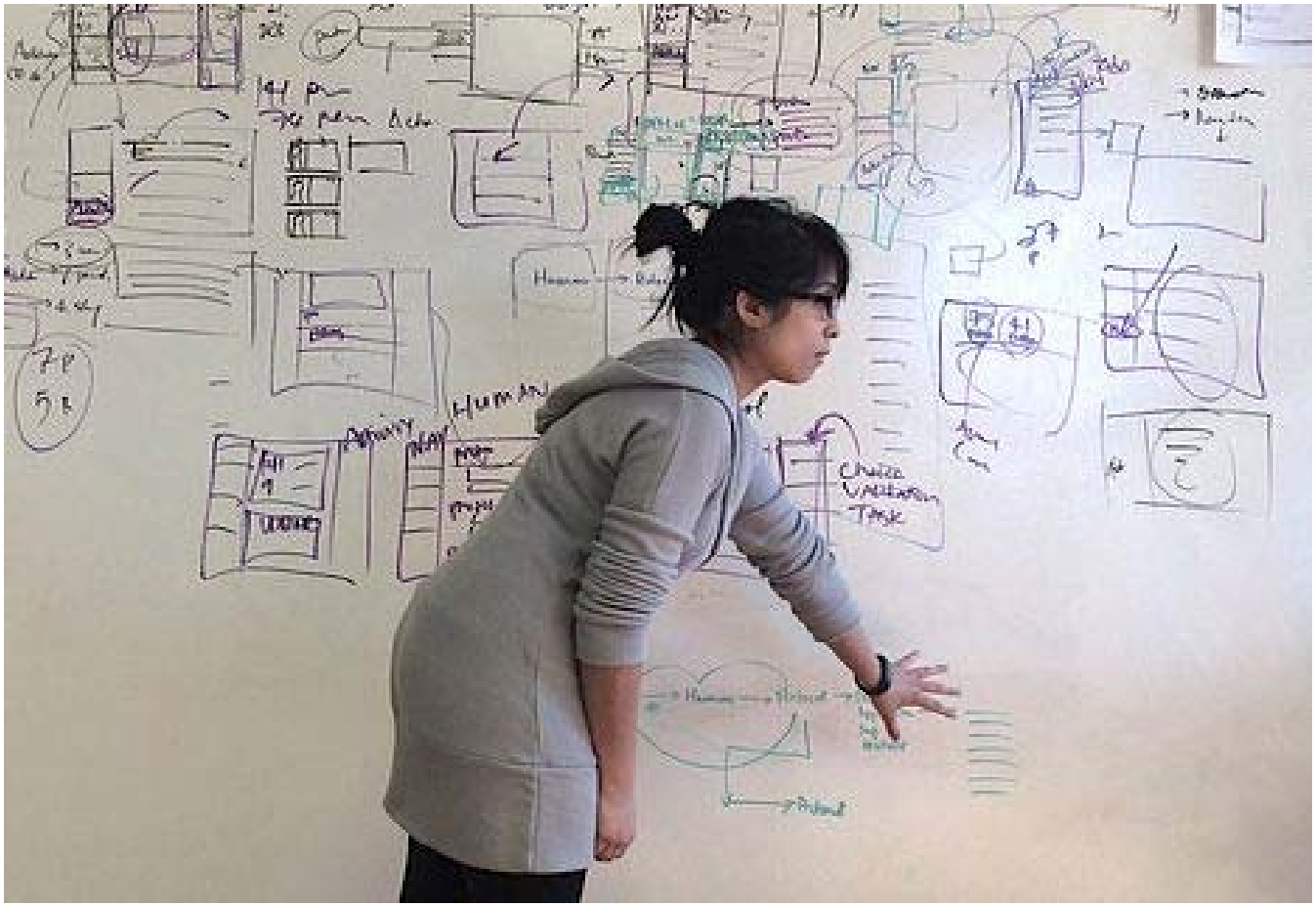
- UML ist die standardisierte Form der visuellen Darstellung von Software.
- Ein Software Engineer muss Anforderungen (Use Cases, User Stories) sowie Software (Struktur, Funktionsweise) visuell zu anderen im Team oder zu Außenstehenden kommunizieren können.
- Code ist aufgrund des Umfangs dafür oft ungeeignet.
- Code bekommt eine bessere Struktur, wenn vorher alle im Team „die gleichen Bilder im Kopf haben“.
- Das ist auch in der Abschlussarbeit wichtig.

3.1 Schlechte Beispiele

...versuchen Strukturen (Komponenten) und Abläufe in einem Bild zu zeigen (Abb. aus einer Abschlussarbeit).



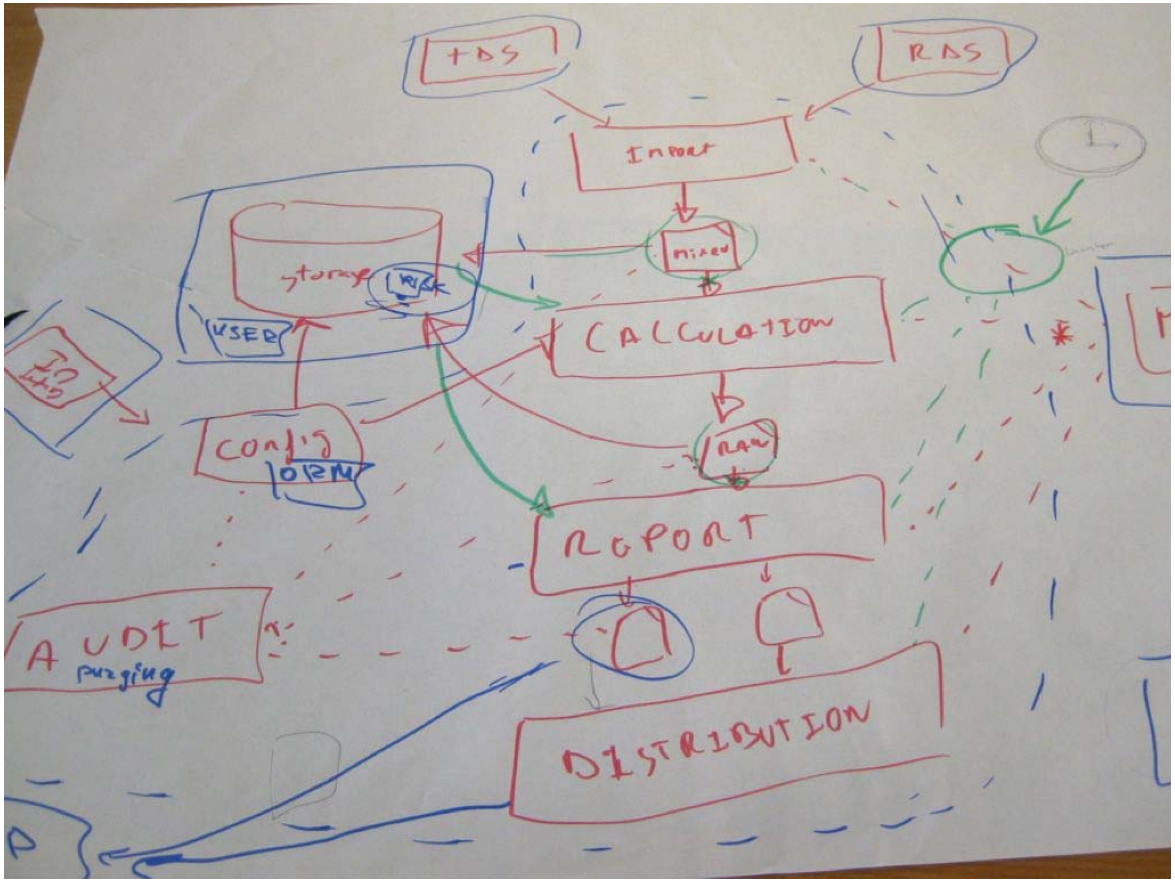
3.2 Schlechte Kommunikation von Software



Whiteboards werden gern genutzt, um Software-Strukturen im Team zu erklären.

Auch hier sollten Regeln beachtet werden, um den Eindruck zu hinterlassen, dass der/die Vortragende die Grundprinzipien kennt und professionell auftreten kann.

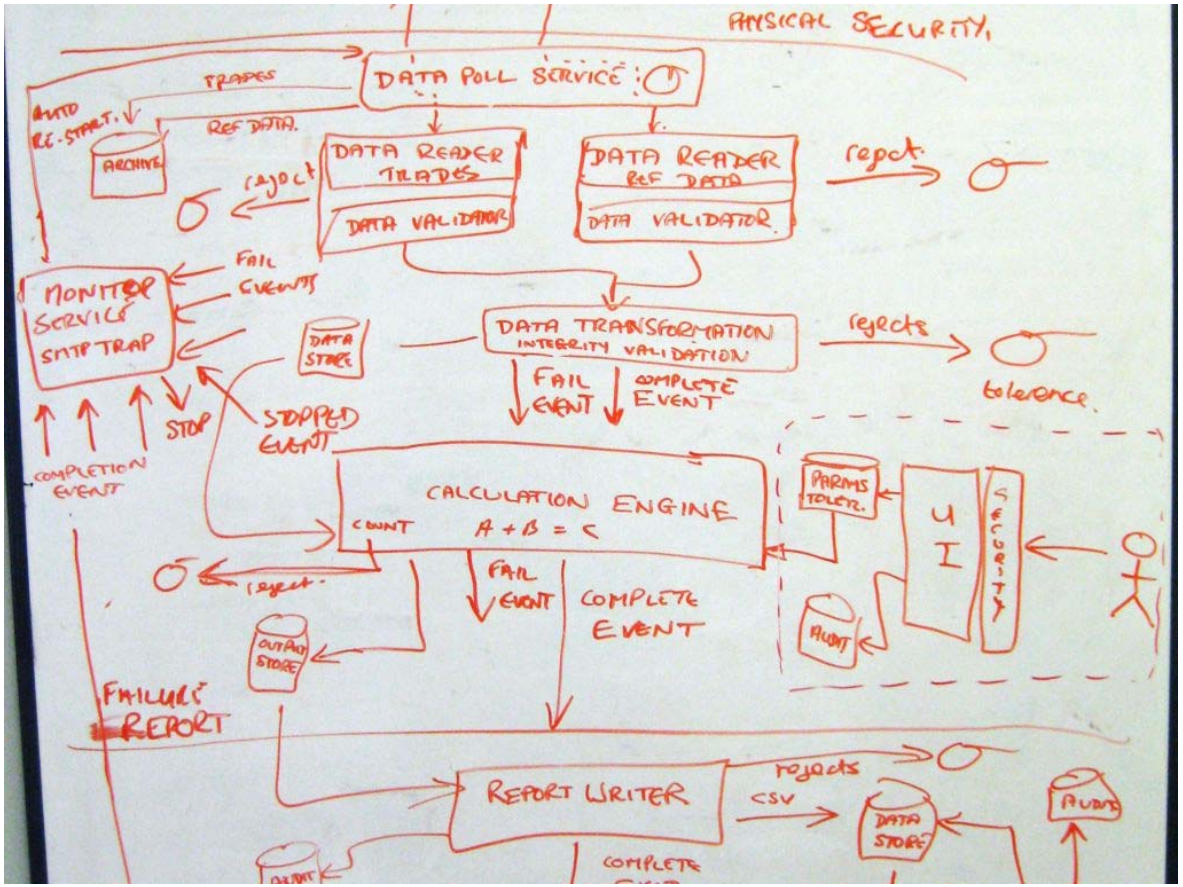
3.2.1 Schlechtes Beispiel (1)



Rating: Schlimm(st)

- **Keine Struktur** im Bild erkennbar (und wohl auch nicht im Kopf des Autors).
  - Keine geraden Linien.
  - Ellipsen, Kreise !
- **Begriffliche Verwirrung**: AUDIT, DISTRI-BUTION, CALCULATION sind Vorgänge (Prozesse). REPORT ist ein Objekt.
  - Nie verschiedene Dinge (Abläufe, Prozesse vs. Objekte) mit gleichen Symbolen darstellen.
  - Begriffe immer klar wählen:
  - Ist ein **Report** (Objekt) oder ein **Report-Generator** (Komponente) oder **Reporting** (Ablauf, Prozess) gemeint?
- Was bedeuten die Farben, Pfeile, Linienarten?

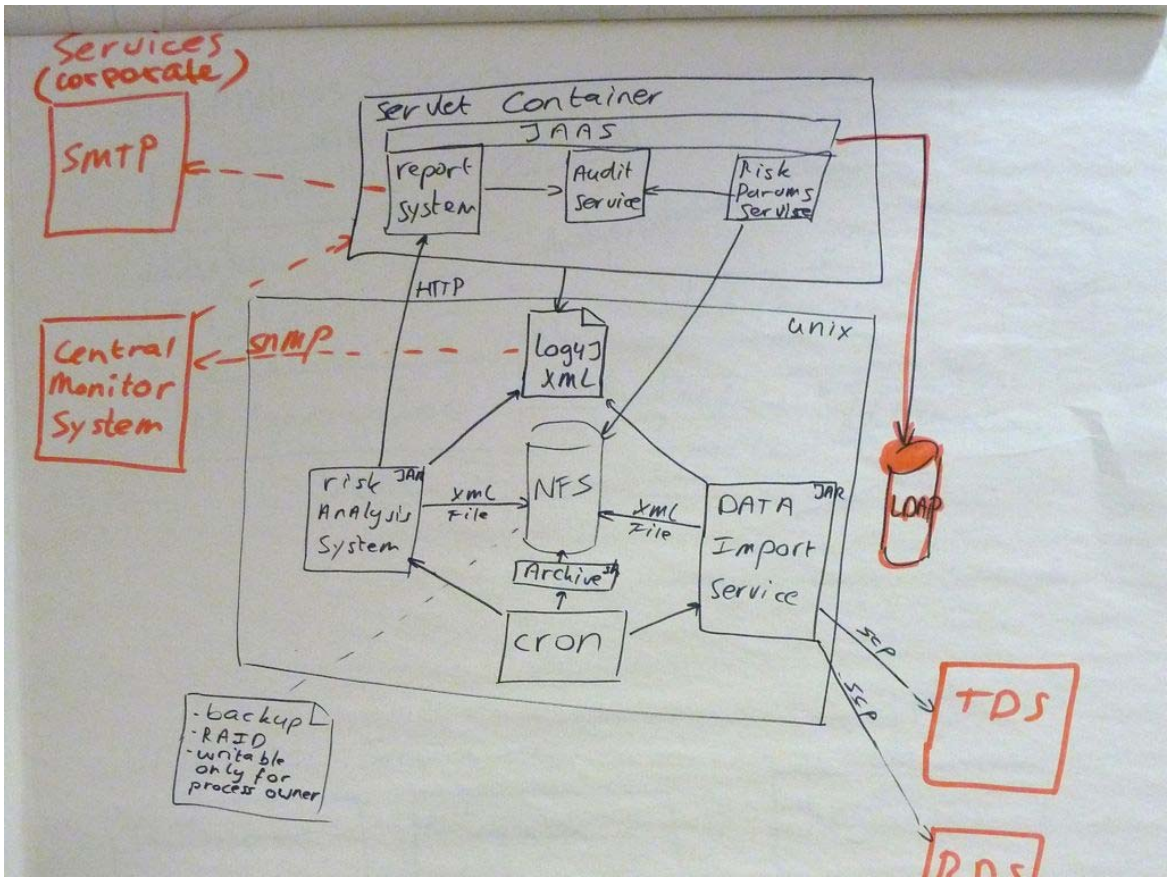
3.2.2 Schlechtes Beispiel (2)



Rating: schlecht

- **Strukturen** werden erkennbar, bleiben aber trotzdem unklar:
  - Was gehört wohin?
  - Begrenzungen (Was ist innen/aussen)?
  - Wo ist ein Anfang? Wo das Ende?
- Zu viel Detail auf einer Ebene
  - Was sind die wichtigen Komponenten?
  - Diese zuerst zeichnen.
  - Dann diese in weiteren Darstellungen verfeinern.
- Aber: gute Begriffswahl,
  - Für Komponenten-Namen: *-Engine, *-Service, *-Store, *-Validator, *-Writer

3.2.3 Besseres Beispiel (3)

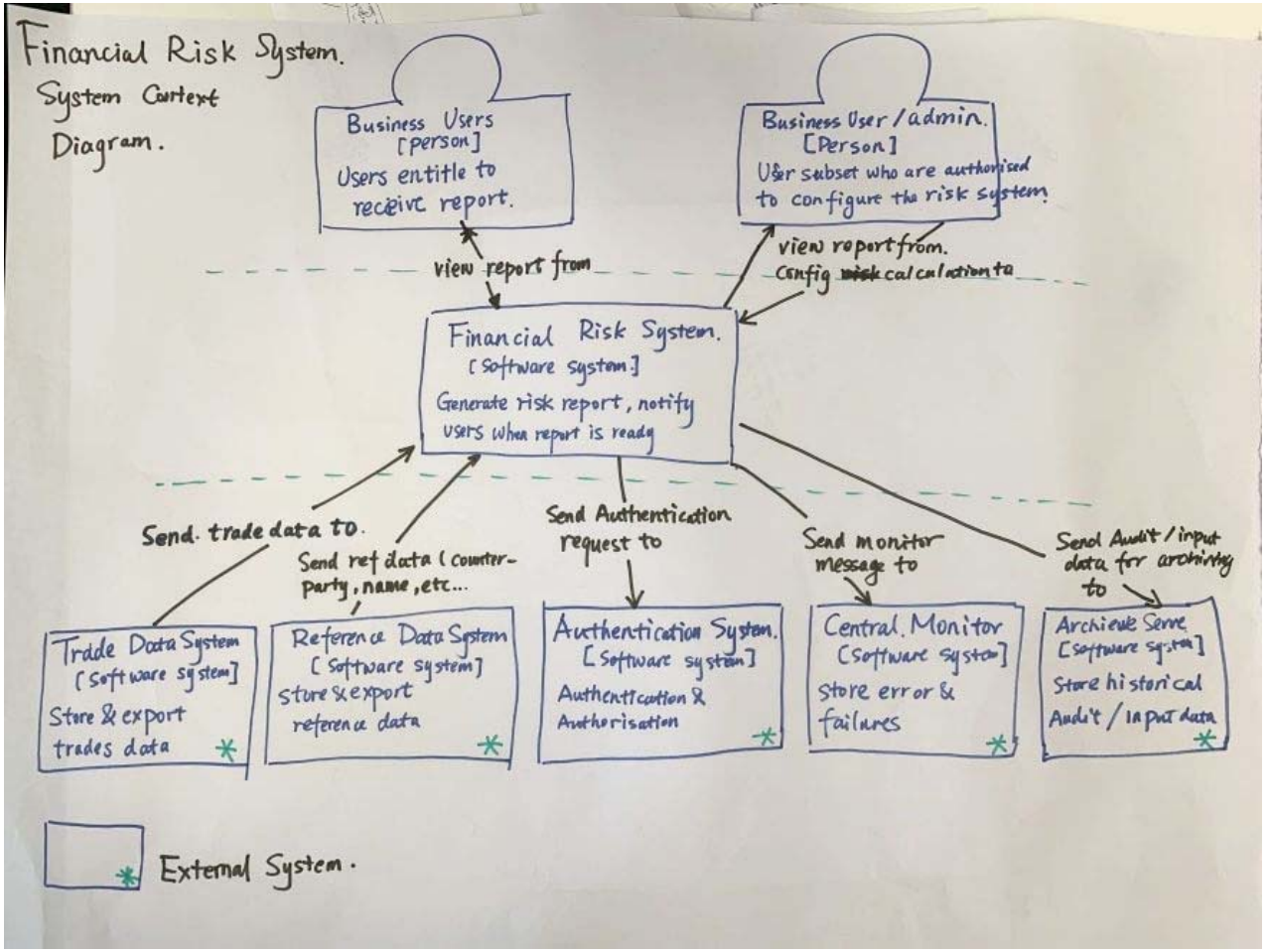


Rating: besser

Erster Eindruck: Autor hat sich Mühe gegeben, möchte klare Struktur des Systems vermitteln.

- Struktur ist erkennbar,
  - Begrenzungen sind erkennbar (innen/aussen).
- Konzepte (Begriffe) sind verwirrt:
  - SMTP ist ein email Protokoll,
  - Central Monitor System ein System.
- Strukturelle Inkonsistenz:
  - Warum ist NFS im System, LDAP ausserhalb?
  - cron ist ein Unix-Dienst, logging.xml eine Datei.
- Die Frage bleibt: kennt der Autor Grundprinzipien?

3.2.4 Ähnliches Beispiel (zur Diskussion)

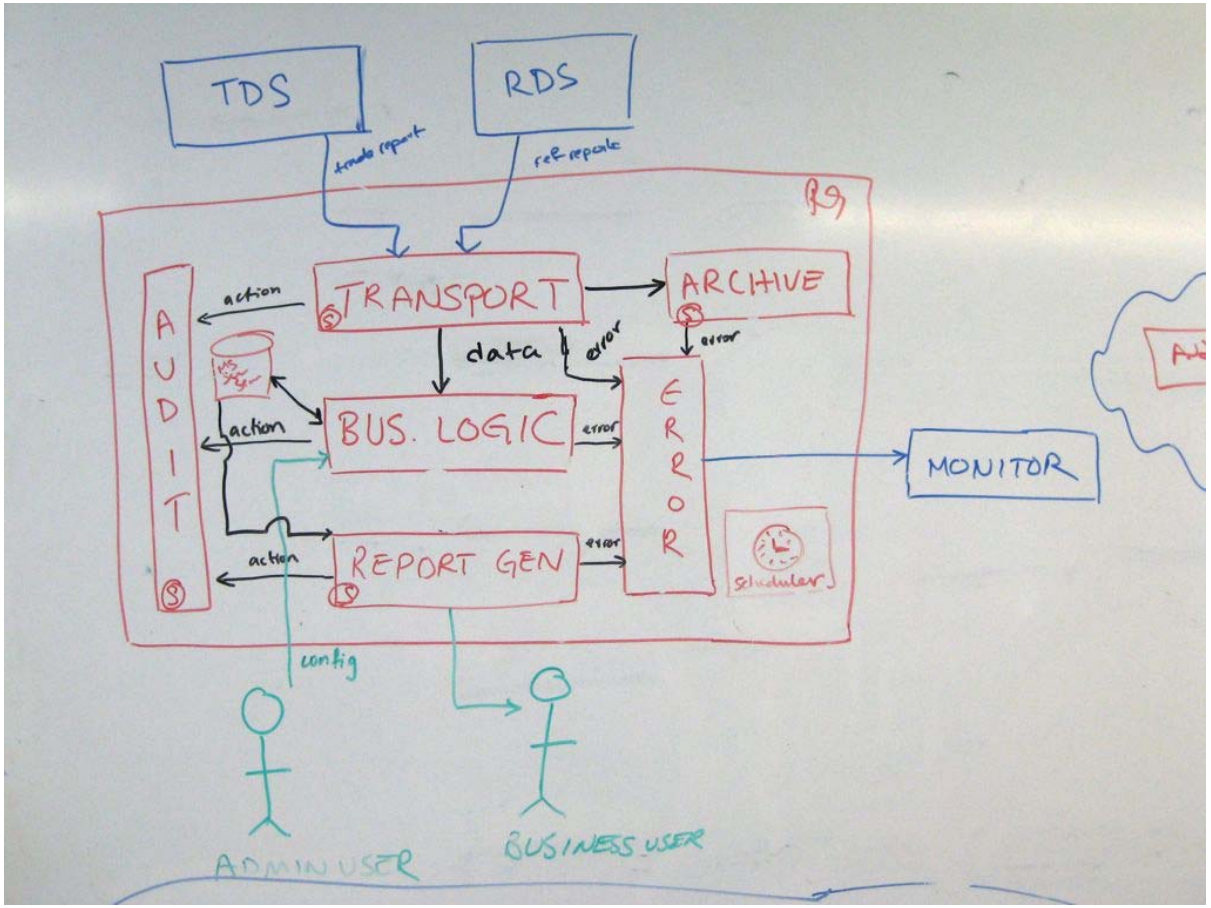


Rating: besser

Was kann hier gesagt werden?

- Strukturen erkennbar?
  - Begrenzungen?
- Klarheit in Begriffen?
  - Verweisen die Namen in den Boxen auf Dinge dergleichen Art?
- Klarheit in den Beziehungen?
  - Was bedeuten die Pfeile?

3.2.5 Ähnliches Beispiel (zur Diskussion)



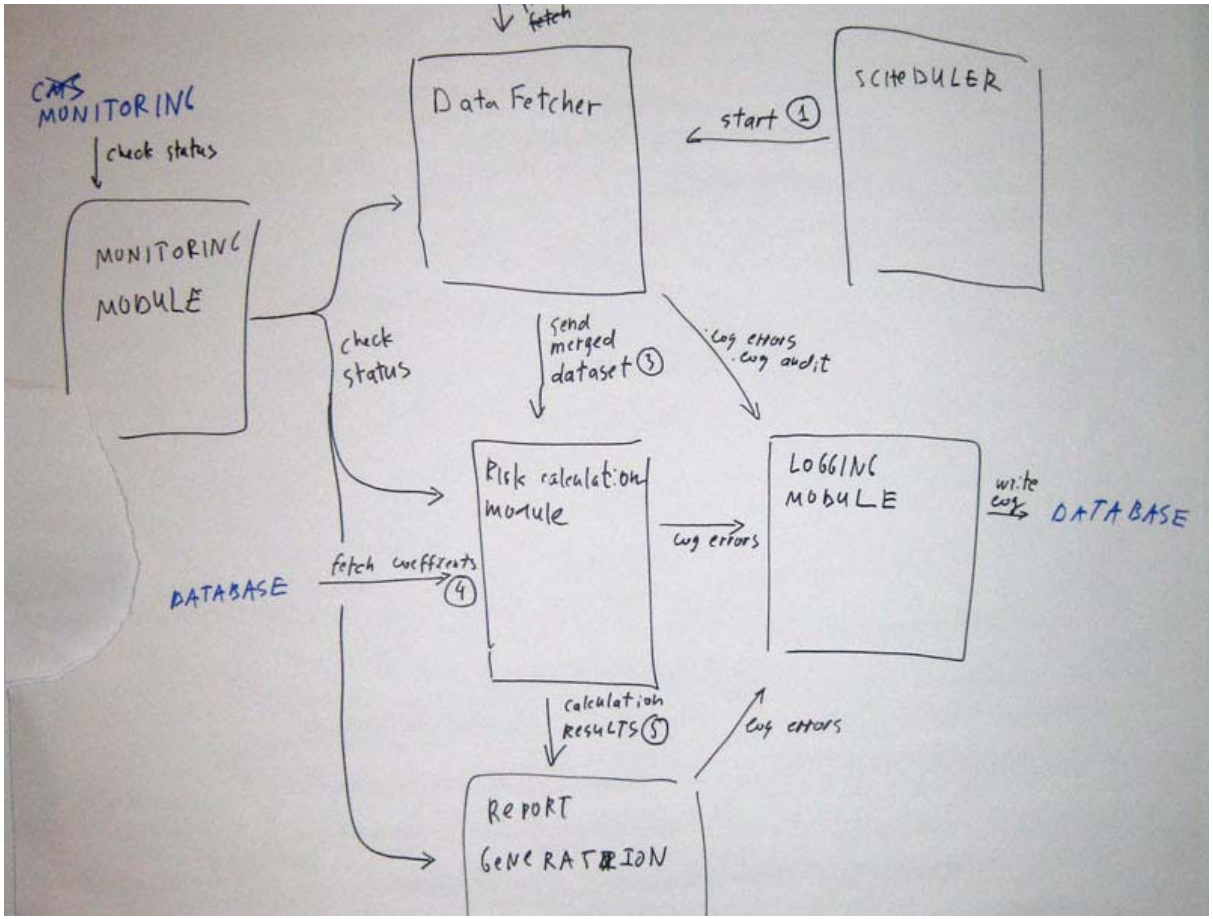
Rating: besser (*)

Ein „professioneller“ Eindruck entsteht, wenn **Symbole von UML** verwendet werden (hier die Aktoren (grün)).

Was kann sonst gesagt werden?

- Strukturen erkennbar?
- Klarheit in Begriffen?
- Klarheit in den Beziehungen (Pfeile)?

3.2.6 Ein fast gutes Beispiel



Rating: fast gut

Die Symbolik erinnert an UML Komponenten Diagramm (Vierecke mit gut gewählten Namen).

Was fehlt:

- Pfeile sollten durch Interface-Symbole und Benutzungsbeziehungen ersetzt werden.
- Begrenzung fehlt (was ist im und was ist außerhalb des Systems).

4 UML - Unified Modelling Language



UML – Unified Modelling Language – ist eine standardisierte, graphische Beschreibungssprache zur Modellierung unterschiedlicher Aspekte von Softwareeigenschaften.

- OMG (Objekt Management Group) ist das Gremium der UML Standardisierung.
- UML 2.5 Spec (2012) ist die aktuelle Version, https://www.omg.org/spec/UML/2.5 .
- Online Ressourcen:
  - Online Referenz: http://www.uml-diagrams.org/uml-25-diagrams.html ,
  - Cheat Sheet: https://www.oose.de/wp-content/uploads/2012/05/UML-Notations%C3%BCbersicht-2.5.pdf.

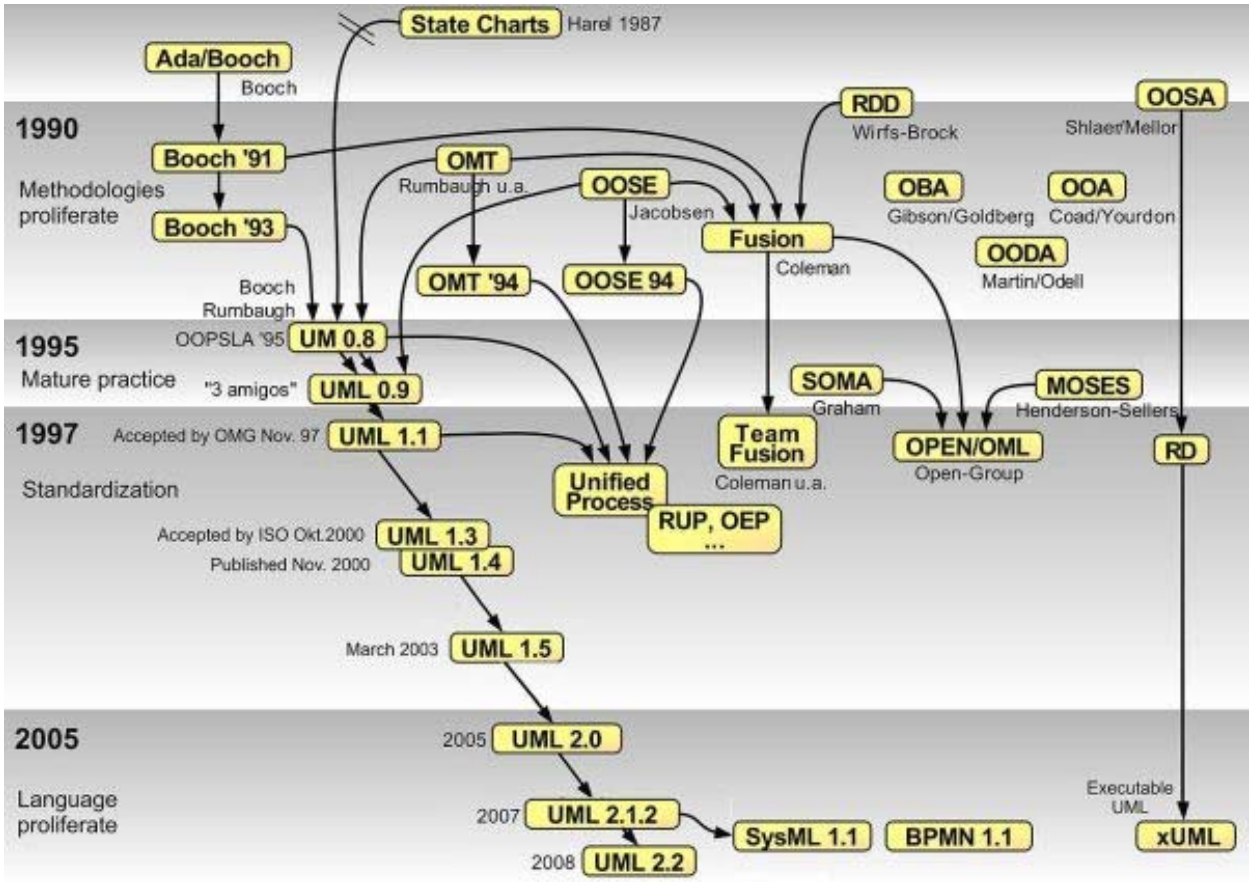
4.1 Ursprünge von UML



Die Entwicklung von UML ist eng verbunden mit:

- **Objektorientierter Programmierung.**
  - Simula67 – Ole-Johan Dahl, Kristen Nygaard, Norwegian Computing Center Oslo, 1965.
  - SmallTalk – Adele Goldberg, Alan Kay, Dan Ingalls, Xerox PARC, Palo Alto, 1972.
  - Objekte, Nachrichten, Klassen, Vererbung.
- **Objektorientierten Methoden des Software-Engineerings – OOSE**.
  - **Grady Booch** (Ada, Booch Method 1993, RUP), **Ivar Jacobson** (OOSE – Object-Oriented Software Engineering).
- **Objektorientierter Modellierung – OMT.**
  - **James Rumbaugh** (OMT - Object-Modelling Technique).
- **UML** entstand aus der Zusammenführung der bis dahin existierenden Methoden.
  - UML 0.8 – 1995. UML 2.5 – 2013.

4.2 Einflüsse auf die Entwicklung von UML



- Bis in die 1990‘er Jahre gab es eine Vielzahl von Ansätzen, Methoden und Vorgehensweisen zur Analyse, Planung und Modellierung von Softwareeigenschaften.
- UML führte diese Vielfalt zusammen und vereinheitlichte die Software-Modellierung.
- **UML ist heute der Standard für Software-Modellierung**.

4.3 Zwei Schulen: OMT + OOSE


Grady Booch


Ivar Jacobson


J. Rumbaugh

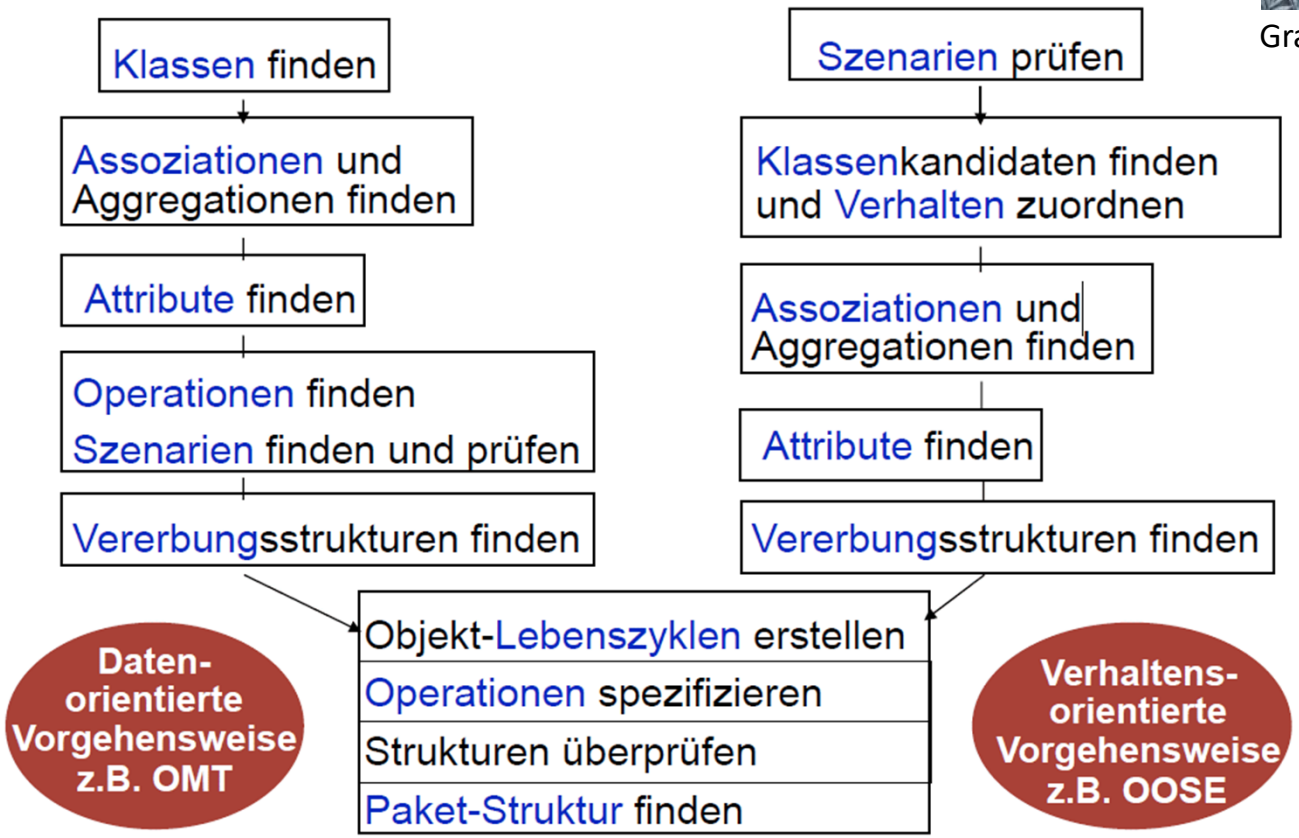
- **Object-Modelling Technique (OMT)**, James Rumbaugh verfolgte den Ansatz, in einem herzustellenden System zuerst Objekte zu identifizieren und als Klassen zu beschreiben. Da Klassen im Mittelpunkt stehen, nennt man es objekt- oder auch daten-orientiertes Vorgehen.
- **Object-Oriented Software Engineering (OOSE)**, G. Booch, Ivar Jacobson
  geht dagegen von einem gewünschten Verhalten aus (Szenarien, Anwendungsfälle) und versucht dann, das System in Komponenten/Klassen zu gliedern und Funktionen diesen Komponenten/Klassen zuzuordnen (Schnittstellen).
- Bei OOSE beginnt man mit der Modellierung von Szenarien und Anwendungsfällen bevor man Klassen modelliert. Daher nennt man es verhaltens-orientiertes Vorgehen.

4.4 Übersicht: OMT + OOSE


Grady Booch

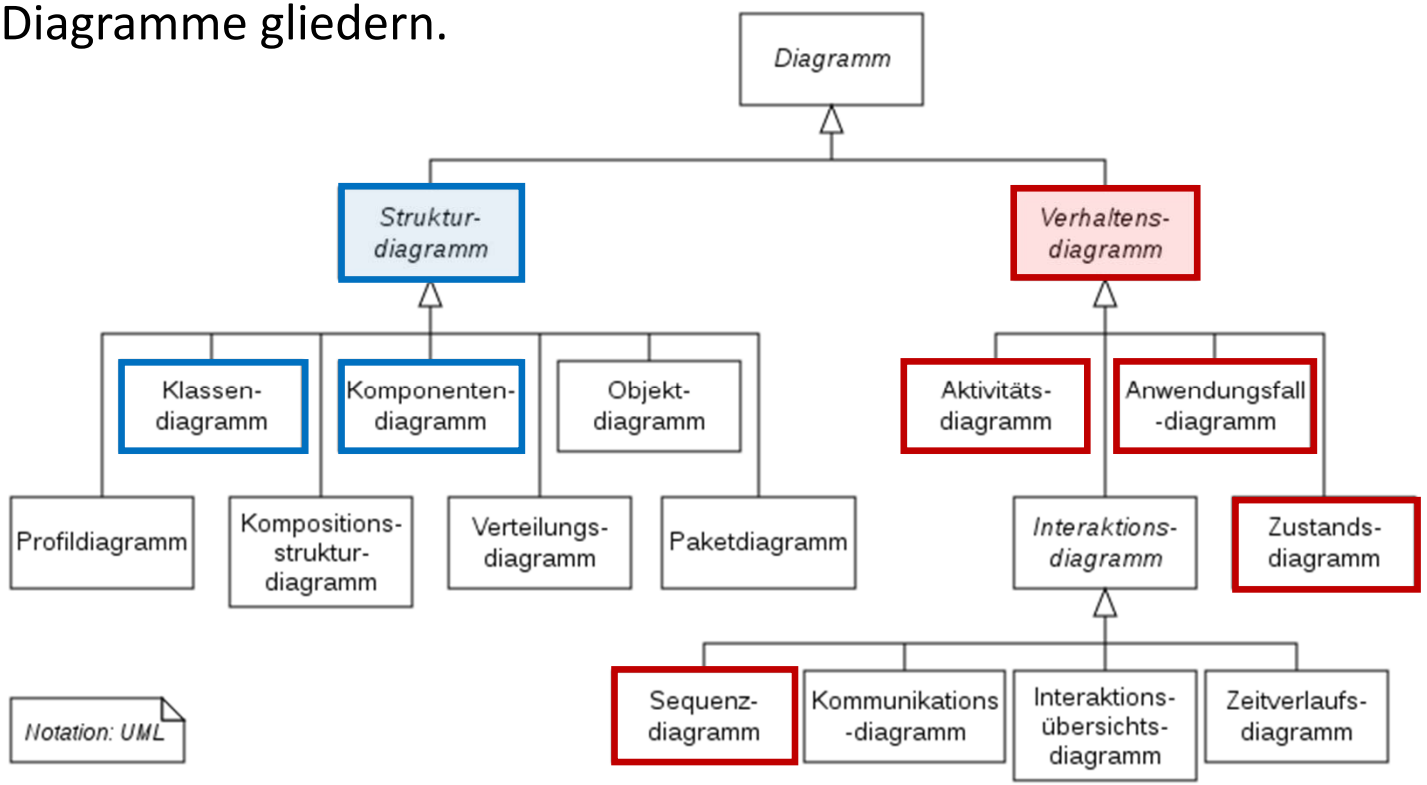

Ivar Jacobson


J. Rumbaugh



5 UML – Diagramm-Arten

UML besteht aus 14 Diagramm-Arten, die sich in Struktur- und in Verhaltens-Diagramme gliedern.

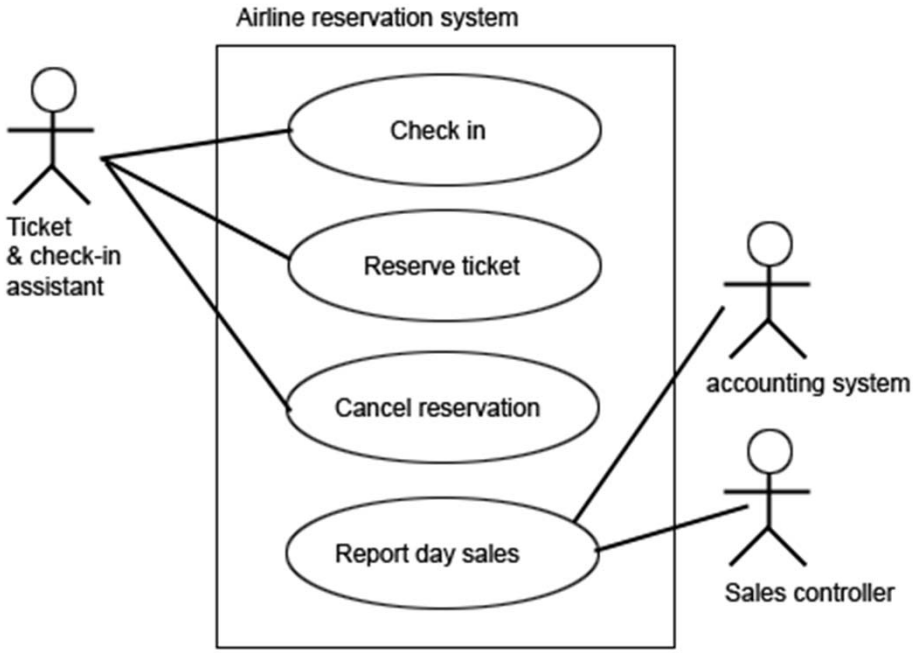


5.1 UML Use Case Diagramm (Anwendungsfälle)

Zeigt **Intentionen** von

Akteuren mit einem

System: wer will was?



5.2 UML Klassendiagramm

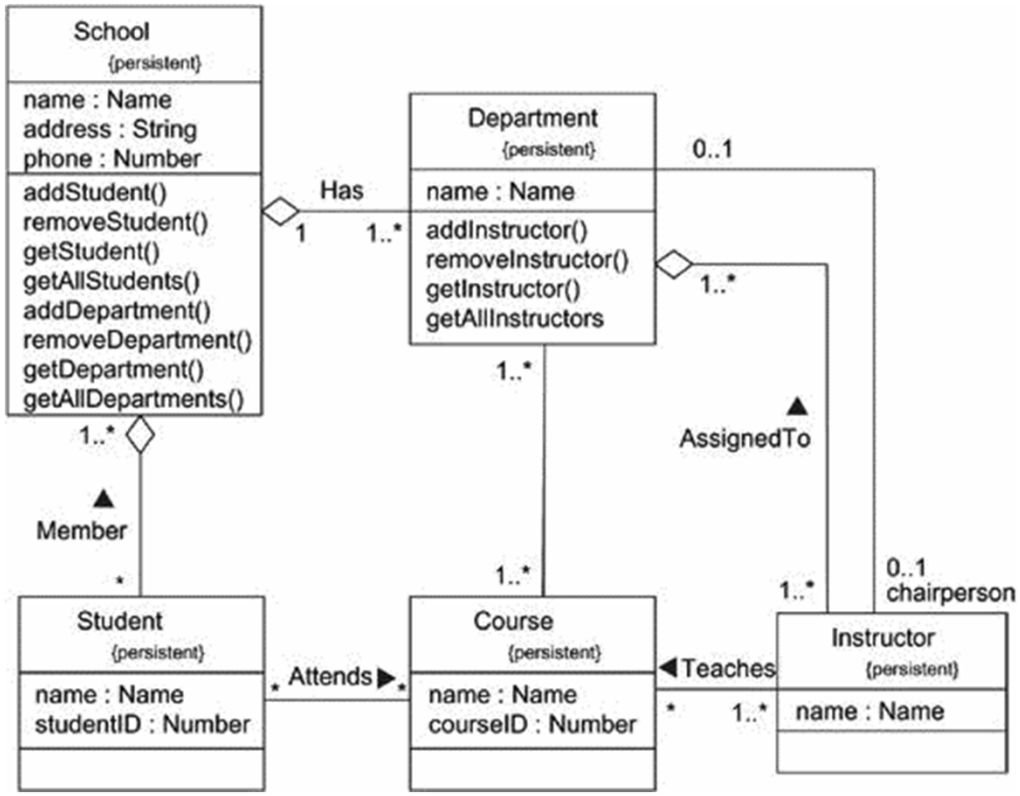
Zeigt die statische

Klassen-Struktur,

sowie **Beziehungen**

zwischen Klassen mit

Kardinalitäten [0, 1, *].



5.3 UML Komponentendiagramm

...stellt die Struktur des Systems mit seinen Bestandteilen, Schnittstellen und Verbindungen dar.

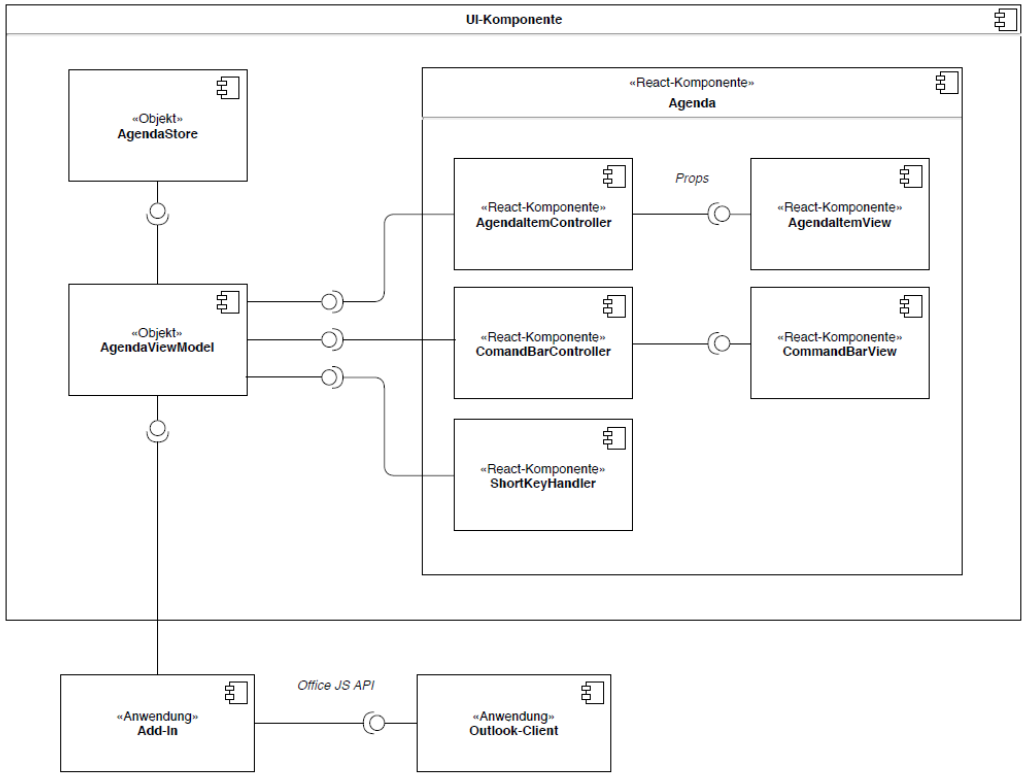


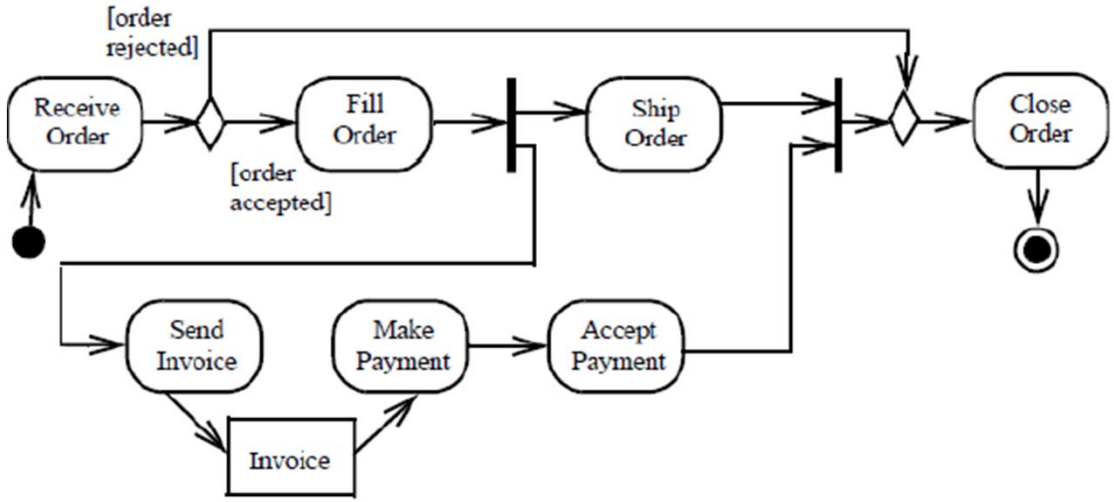
Abbildung 20: UML-Komponentendiagramm des umgesetzten MVVM-Entwurfsmusters

5.4 UML Aktivitätsdiagramm

Zeigt den dynamischen

Ablauf eines **Workflows**

oder **Steuerflusses**.

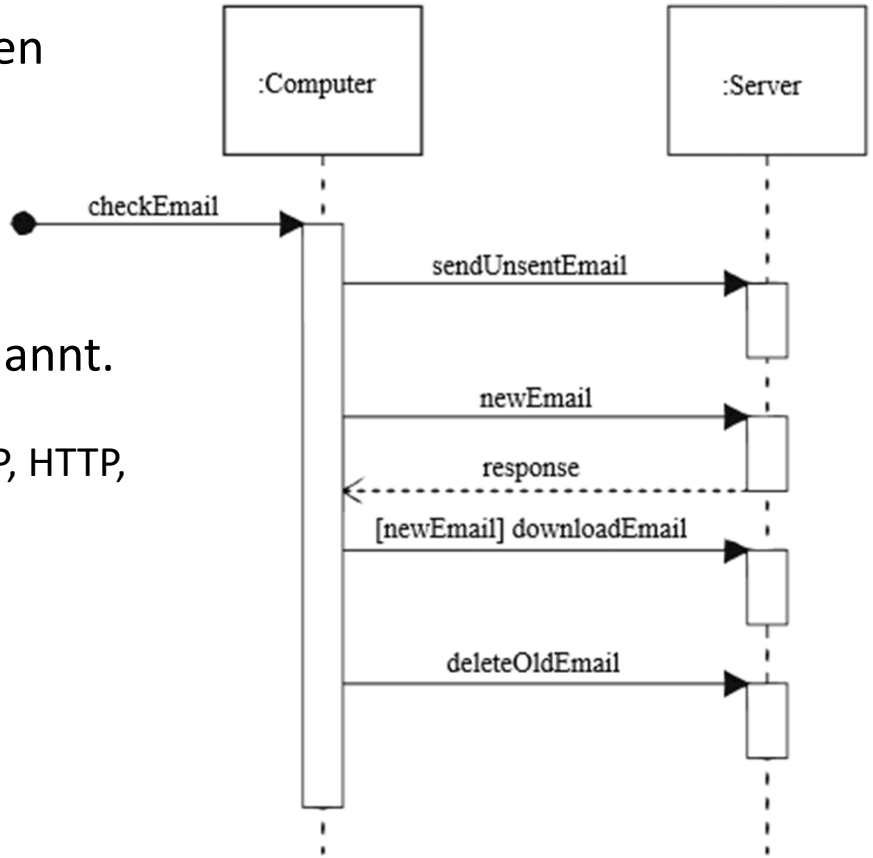


5.5 UML Sequenzdiagramm

Zeigt die **Interaktionen** der Komponenten eines Systems.

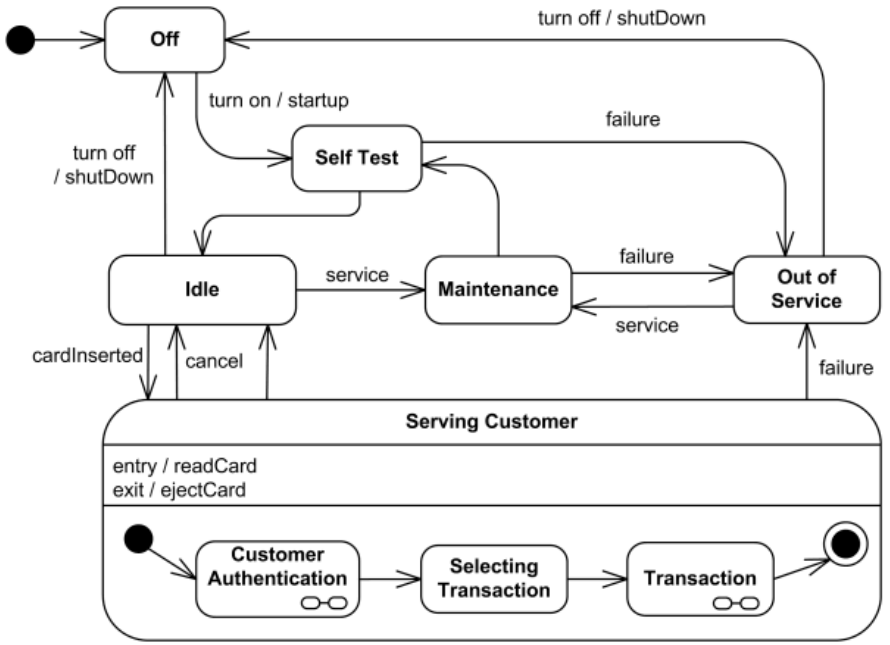
Ablaufregeln für den Austausch von Nachrichten werden auch **Protokoll** genannt.

- Bekannte Protokolle: IEEE 802.11, IP, TCP, UDP, HTTP, IMAP, FTP, SSH Connection Protocol – siehe: http://ietf.org).



5.6 UML Zustandsdiagramm

Beschreibt das Verhalten eines Systems als **Zustandsautomaten** mit Zuständen von Objekten, Ereignissen, Funktionen und resultierenden Folgezuständen.

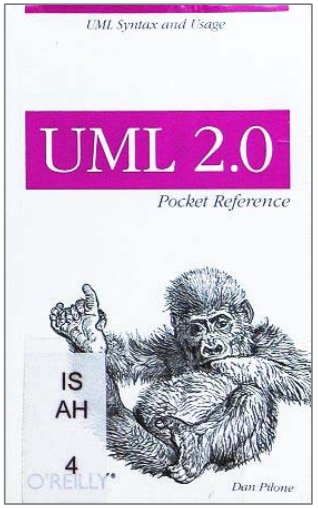


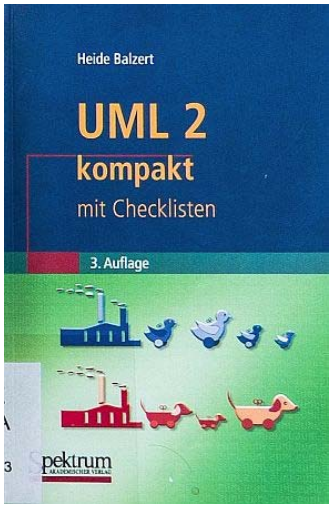
Formal ist ein Zustandsautomat ist ein 5-Tupel (Σ, S, s0, δ, F) mit:

- Σ ist das Eingabealphabet (eine endliche, nicht leere Menge von Symbolen),
- S ist eine endliche nicht leere Menge von Zuständen,
- s0 ist der Anfangszustand und ein Element aus S,
- δ ist die Zustandsübergangsfunktion: δ: S x Σ → S,
- F ist die Menge von Endzuständen und eine (möglicherweise leere) Teilmenge von S.

6 UML Ressourcen

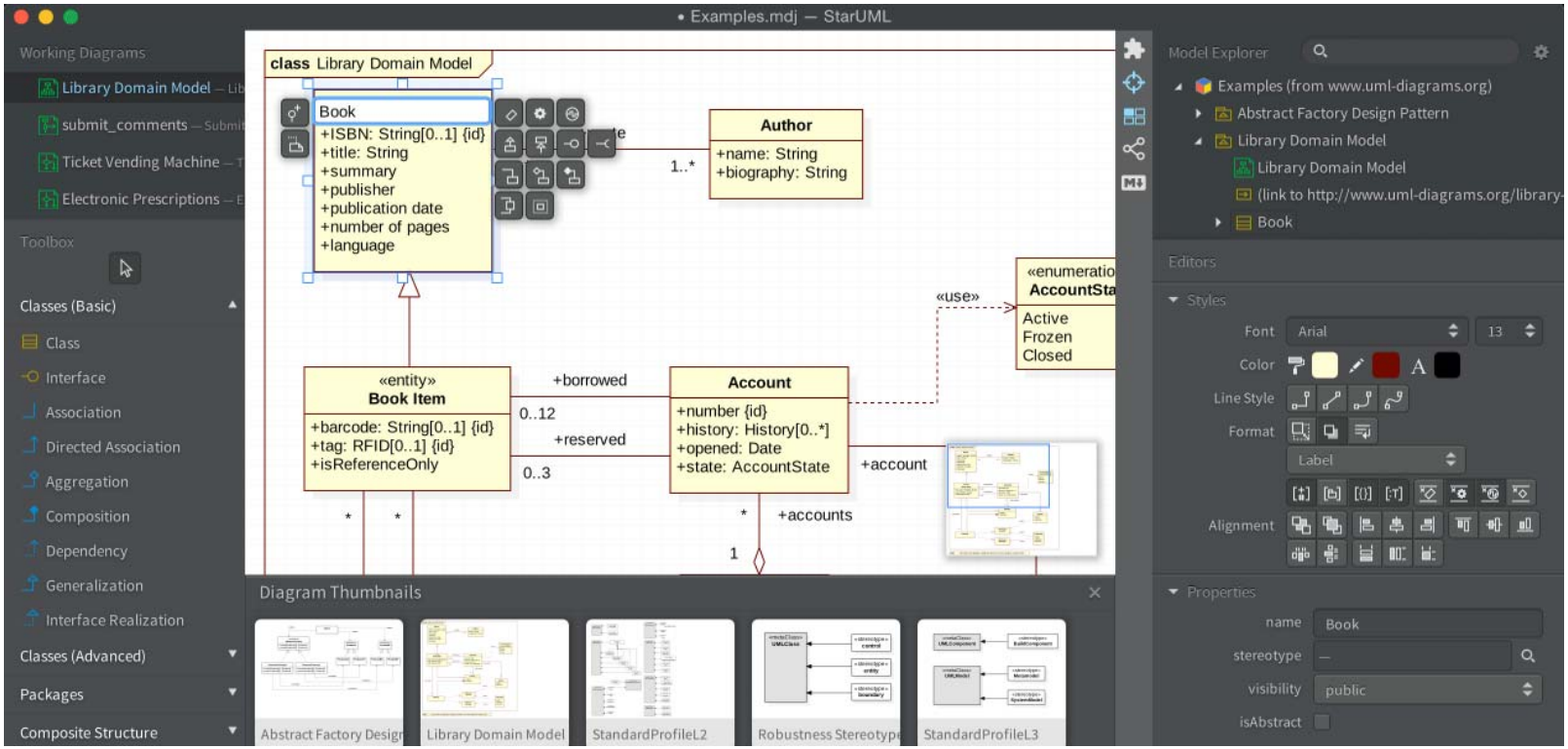
- UML 2.5 Online Referenz:
  - http://www.uml-diagrams.org/uml-25-diagrams.html .
- UML 2.5 Kurzreferenz, Cheat Sheet:
  - https://www.oose.de/wp-content/uploads/2012/05/UML-Notations%C3%BCbersicht-2.5.pdf.
- Bücher aus der Bibliothek:





6.1 UML Werkzeuge, UML Designer

StarUML ist in gutes Werkzeug, um UML Diagramme zu erstellen, siehe: http://staruml.io , Alternativen: https://www.guru99.com/best-uml-tools.html .



Zusammenfassung

1. Motivation
   - Gute Bsp. aus einer Abschlussarbeit
2. Modellbildung, Modellierung von Software
3. Visuelle Darstellung von Software
   - Schlechte Beispiele
4. UML – Unified Modelling Language
   - Ursprung, Entwicklung, zwei Schulen OOSE, OMT.
5. UML Diagramm-Arten
   - Strukturdiagramme: Klassen-, Komponenten-Diagramm.
   - Verhaltensdiagramme: Use Case-, Aktivitäts-, Zustands-, Sequenz-Diagramm.
6. UML Ressourcen